

Super Factory Manager programming guide

SFML reference and practical handbook · Super Factory Manager 4.34

AI disclosure: This guide was generated by AI using SFM 4.34 source code and official examples. It is an independent guide, not official SFM documentation. See [validation details](#) for the checks performed and their limits.

SFML is the language you write on a Super Factory Manager disk to move items, fluids, energy, and supported chemicals between labelled blocks. This guide teaches SFML in SFM 4.34, with a complete keyword reference, original examples, and explanations of how transfer rules work. It is independent of any particular modpack.

Start with **Your first program**, then **How execution works** and **Amounts, RETAIN, and EACH**. Use the **Keyword reference** when reading somebody else's script. The separate `examples` folder contains ready-to-edit programs. The HTML edition has search, navigation, copy buttons, and a print layout.

Version scope: examples and language behavior are checked against SFM **4.34.0 for Minecraft 1.21.1 / NeoForge**, source tag `4.34.0-1.21.1`, commit `f5366c79c823ff52712130e69dd9c8166c70bd14`. Resource integrations can differ between Minecraft ports; port-specific details are identified where relevant. Syntax checks use that release's lexer and parser and do not establish live-machine behavior. The project's development branch may contain newer features. The language is defined by the [release grammar](#).

1. Your first program

Build a small test network

You need a **Factory Manager**, **Program Disk**, **Label Gun**, Inventory Cables, and two chests. Use a recipe viewer such as JEI to check the crafting recipes in the current installation.

1. Place the Factory Manager and connect both chests to its cable network.
2. Put a few inexpensive items in the first chest. Leave room in the second.
3. Open the Label Gun's interface and create/select `source`. Apply it to the first chest. Create/select `destination` and apply it to the second.
4. **Push the Label Gun's labels to the Factory Manager.** Labels stored only on the gun are not enough. Follow the gun's displayed push/pull controls; pulling reads an existing manager's assignments, while pushing writes assignments to it.
5. Open the Program Disk's editor, paste the program below, save it, and put the disk in the manager. A disk can also be edited while held in your hand.
6. Check the manager's compilation/problems display and running state. At normal server speed, items should begin arriving within about a second.

```
NAME "My first transfer"
```

```
EVERY 20 TICKS DO
```

```
    INPUT FROM source
```

```
    OUTPUT TO destination
```

```
END
```

Line	Meaning
NAME "My first transfer"	Optional human-readable program title.
EVERY 20 TICKS DO	Start a trigger that runs every 20 game ticks.
INPUT FROM source	Make items in the blocks labelled <code>source</code> available to later output statements.
OUTPUT TO destination	Transfer eligible items into blocks labelled <code>destination</code> .
END	Close the trigger.

A missing resource filter defaults to **items**, so this program does not also move fluids or FE. Labels are names you assign; `source` and `destination` have no special built-in meaning. The official [basic example](#) uses the same input/output model.

If nothing happens

Check the labels **in the manager**, cable connectivity, whether both chunks are loaded, destination space, and the problems panel. For an ordinary chest, start without a side or slot filter. Add filters only after this simple transfer works.

2. How execution works

SFML is a small automation language with scheduled triggers. It does not provide ordinary variables, functions, arithmetic expressions, `for` loops, or `while` loops in this release.

When a trigger runs, its statements execute in order:

```
Trigger becomes due
```

```
↓
```

```
INPUT registers eligible source slots and limits
```

```
↓
```

```
OUTPUT matches sources with destinations and transfers resources
```

```
↓
```

```
Later OUTPUT statements can use the remaining active inputs
```

```
↓
```

```
FORGET clears selected inputs, or the trigger finishes
```

INPUT is not a physical buffer. Merely inputting from a chest does not remove its contents. Transfers happen when an output can accept them. Resources that cannot be inserted normally stay in their source, subject to the participating mods implementing their inventory interfaces correctly.

An **INPUT** remains active for later **OUTPUT** statements in that trigger. Its transfer allowance is shared across those outputs. New input statements add more sources; they do not replace old inputs. Use `FORGET` between unrelated jobs.

Each trigger runs with a fresh execution context. Inputs do not carry over to the next trigger or timer run. World changes do persist: after an output inserts items, a later condition can see the new inventory state. Separate machines can consume or create items between runs.

These execution details follow [InputStatement](#), [OutputStatement](#), and [Program](#).

Priority comes from separate output statements

```
NAME "Keep supplies before storing overflow"

EVERY 20 TICKS DO
  INPUT minecraft:iron_ingot FROM incoming
  OUTPUT RETAIN 64 minecraft:iron_ingot TO workshop
  OUTPUT minecraft:iron_ingot TO warehouse
END
```

First, top up `workshop` to 64 matching ingots. Then send what is still available to `warehouse`. A combined destination list is not a substitute for an explicit priority policy; internal block ordering is not a stable layout contract.

3. Writing syntax correctly

Topic	Rule
Keywords	Case-insensitive: <code>INPUT</code> , <code>input</code> , and <code>Input</code> are equivalent.
Labels	Use exact spelling and case. <code>ore_bin</code> and <code>Ore_Bin</code> are different names.
Resource IDs	Use the actual registry IDs, normally lowercase, such as <code>minecraft:iron_ingot</code> .
Comments	<code>--</code> starts a comment through the end of the line. <code>#</code> introduces a tag, not a comment.
Whitespace	Spaces, tabs, and line breaks separate tokens. Indentation is for people.
Statements	No semicolons. No braces. Keywords mark statement boundaries.
Numbers	Nonnegative whole decimal numbers: <code>64</code> , <code>1000</code> , <code>100000</code> . No <code>1k</code> , decimal fractions, negative quantities, or <code>64*9</code> .
Strings	Double quotes: <code>"fuel storage"</code> . They allow spaces or otherwise awkward names.
Commas	Separate labels, resource-limit entries, sides, and slot selections in their respective positions.
Blocks	<code>EVERY ... DO ... END</code> and <code>IF ... THEN ... END</code> . Every nested block needs its own <code>END</code> .
Reserved words	Quote a label that conflicts with a keyword, e.g. <code>"input"</code> .
Label length	Maximum 256 characters in this release.

Prefer plain lowercase labels with underscores: `ore_input` , `fuel` , `smelters` , `finished` . Asterisks in a resource filter are patterns; do not assume they expand label names. Assign the same label to several blocks to form a group. A block can have several labels, but overlapping groups can make routing and counting harder to reason about. These rules come from the [grammar](#), [label lookup](#), and [AST builder](#).

Two equivalent statement orders

```
EVERY 20 TICKS DO
  INPUT 16 minecraft:cobblestone FROM source
  OUTPUT TO destination
END
```

The language also accepts `FROM source INPUT 16 minecraft:cobblestone` and `TO destination OUTPUT` . This guide uses the first style consistently.

4. Trigger timing

Form	Meaning
<code>EVERY 20 TICKS DO</code>	Run once every 20 game ticks.
<code>EVERY 1 SECOND DO</code>	Same interval as 20 ticks.
<code>EVERY 5 SECONDS DO</code>	Run every 100 ticks.
<code>EVERY 20 GLOBAL TICKS DO</code>	Use the world's game-time clock instead of the manager's local clock.
<code>EVERY 20 GLOBAL PLUS 5 TICKS DO</code>	Run when world game time modulo 20 equals 5.
<code>EVERY 20G+5 TICKS DO</code>	Compact form of the preceding example.
<code>EVERY REDSTONE PULSE DO</code>	Run for redstone pulses received by the manager.

`TICK` / `TICKS` and `SECOND` / `SECONDS` are aliases. `G` abbreviates `GLOBAL` ; `+` abbreviates `PLUS` . An omitted interval number is 1, so `EVERY SECOND` is valid. Prefer writing numbers explicitly.

PLUS is a phase offset, not addition to the repeat interval. `EVERY 20 PLUS 5 TICKS` still repeats every 20 ticks. Use an offset from 0 through interval minus 1. With `SECONDS` , both interval and offset are converted to ticks: `EVERY 5 PLUS 1 SECONDS` repeats every 100 ticks at offset 20. An out-of-range offset can make a trigger never run.

A local clock is tied to the manager and can be reinitialized after unloading. `GLOBAL` is useful for coordinating managers, but it does not make separately loaded managers atomic or guarantee execution order between them. At the normal 20 TPS, 20 ticks is one second of real time; a lagging server stretches that real-time interval.

The shipped defaults allow a minimum **20-tick interval for general transfers** and **1 tick for triggers whose resource I/O is exclusively Forge Energy**. Server configuration can change these limits. Put FE in its own trigger if you need per-tick power delivery. Timing and defaults are documented in the [timer example](#), [Interval implementation](#), and [server configuration](#).

```
NAME "Separate item and energy schedules"
```

```
EVERY 20 TICKS DO
```

```
    INPUT minecraft:coal FROM fuel
```

```
    OUTPUT RETAIN 8 minecraft:coal TO EACH machines
```

```
END
```

```
EVERY TICK DO
```

```
    INPUT fe:: FROM battery
```

```
    OUTPUT fe:: TO machines
```

```
END
```

The generic `machines` label in this example must expose a suitable coal item input and FE input. Adjust sides to the actual machines.

5. Resource IDs, types, and units

A resource filter can specify both **what kind of resource** and **which registry entry** it matches.

Filter	Meaning
<code>minecraft:iron_ingot</code>	Exactly that item ID.
<code>iron_ingot</code>	Items named <code>iron_ingot</code> in any namespace.
<code>*</code>	All items.
<code>mekanism:*</code>	All items in the Mekanism namespace.
<code>*ingot*</code>	Items whose registry path contains <code>ingot</code> .
<code>item::</code>	All items, explicitly stating the type.
<code>fluid::</code>	All fluids.
<code>fluid::water</code>	Fluids named <code>water</code> in any namespace.
<code>fluid:minecraft:water</code>	Exactly Minecraft water.
<code>fe::</code>	Forge Energy.
<code>chemical::</code>	Mekanism chemicals, when its integration is present.
<code>chemical:mekanism:oxygen</code>	Exactly the Mekanism oxygen chemical.
<code>sfm:item:minecraft:iron_ingot</code>	Fully qualified item filter.
<code>sfm:fluid:minecraft:water</code>	Fully qualified fluid filter.

The full layout is:

```
resource_type_namespace : resource_type_name : resource_namespace : resource_path
sfm                      : fluid              : minecraft            : water
```

One-part filters default to the `sfm:item` type and any resource namespace. Two-part filters are ordinary `namespace:path` item IDs. Three-part filters use SFM's type namespace, as in `fluid:minecraft:water`. Four parts specify everything. Empty namespace/path positions act as wildcards in the parsed filter.

`fluid::water` **has two colons for a reason.** `fluid:water` is a two-part *item* ID, not the same fluid selector. These expansions follow the [resource example](#) and [ResourceIdentifier](#).

What the quantities count

Type	Numeric unit	Example
Items	Individual items, not stacks	<code>64 minecraft:cobblestone</code> is 64 cobblestone.
Fluids	Millibuckets (mB) for these Forge/NeoForge handlers	<code>1000 fluid:minecraft:water</code> is one bucket.
Forge Energy	FE	<code>100000 fe::</code> is a transfer allowance of 100,000 FE for that statement execution.
Mekanism chemicals	Amount reported by the chemical handler	Use the machine's chemical display/inspection to choose an amount.
Redstone resource	Signal strength	<code>HAS</code> can inspect a remote signal; see Redstone below.

An amount in a program is not automatically a per-second rate. Moving at most 100,000 FE every tick has a different possible throughput from moving it every 20 ticks. Source extraction, destination insertion, capacity, and availability still constrain transfers.

`fe::`, `rf::`, `energy::`, `power::`, and `forge_energy::` resolve to Forge Energy in this release. In the Minecraft 1.21.1 integration, `chemical::` is preferred; `gas::`, `infusion::`, `pigment::`, and `slurry::` are registered compatibility names backed by the unified chemical handler. Do not assume these names prove a substance is a fluid. The integration is defined in [SFMMekanismCompat](#).

Create rotation, Botania mana, Ars Nouveau source, and arbitrary mod-specific data are not automatically transferable just because SFM can connect to the block. The [resource registry](#) identifies implemented types; a compatible exposed handler is also necessary.

6. INPUT and OUTPUT

Think of the statement shape as:

```
INPUT [resource limits] [EXCEPT resource IDs] FROM [EACH] labels [round robin] [sides] [slots]
OUTPUT [resource limits] [EXCEPT resource IDs] TO [EMPTY SLOTS IN] [EACH] labels [round robin] [sides] [slots]
```

Square brackets here mean optional documentation placeholders. Do not type them into SFML.

Select only what the next operation needs

```
NAME "Ore delivery"

EVERY 20 TICKS DO
    INPUT 32 minecraft:raw_iron FROM ore_input
    OUTPUT minecraft:raw_iron TO crusher
END
```

This permits up to 32 raw iron to move during this trigger execution. It does not manufacture a batch of 32 or require all 32 to be present. With only 7 available and enough destination room, up to 7 can move.

An output filter can further narrow an input, but cannot widen it. `INPUT minecraft:iron_ingot` followed by `OUTPUT minecraft:gold_ingot` has no eligible gold source.

Multiple resources and shared versus separate limits

```
NAME "Mixed delivery"

EVERY 20 TICKS DO
    INPUT
        16 minecraft:iron_ingot,
        8 minecraft:gold_ingot,
    FROM stock
    OUTPUT TO assembly_supply
END
```

Each comma-separated entry has its own limit. A trailing comma is accepted. By contrast, `16 minecraft:iron_ingot OR minecraft:gold_ingot` gives the matching alternatives a **shared allowance of 16**, unless resource-level `EACH` expands it.

Keep resource groups disjoint when you want exact predictable budgets. Overlapping patterns create separate matching rules, rather than automatically becoming one deduplicated policy. See the [filtering example](#) and [resource trackers](#).

7. Amounts, RETAIN, and EACH

These modifiers solve three different questions: how much may move, how much should remain, and whether the rule applies separately to resources or blocks.

Quantity versus stock level

Statement fragment	Effect
<code>INPUT 16 iron_ingot FROM stock</code>	Allow up to 16 matching ingots to be extracted during this execution.

Statement fragment	Effect
INPUT RETAIN 64 iron_ingot FROM stock	Extract only what can be spared while leaving 64 matching ingots in the selected source scope.
INPUT 16 RETAIN 64 iron_ingot FROM stock	Move at most 16 while also preserving that reserve.
OUTPUT 16 iron_ingot TO supply	Insert at most 16 additional matching ingots. Existing stock does not reduce this quantity allowance.
OUTPUT RETAIN 64 iron_ingot TO supply	Top up to a matching stock level of 64. Existing matching stock counts toward it.
OUTPUT 16 RETAIN 64 iron_ingot TO supply	Add at most 16 while stopping at 64 matching stock.

If a destination holds 50 ingots, `OUTPUT 16 RETAIN 64 iron_ingot` can add at most **14**. If it already holds 80, this output adds **0**; it does not remove the extra 16. If a source holds 70, `INPUT 16 RETAIN 64 iron_ingot` makes at most **6** available.

Retention counts the matching resource/filter in the selected accessible sides and slots. It is not necessarily a count of every compartment visible in the machine's GUI. The behavior comes from the [input retention tracker](#) and [output retention tracker](#).

The four uses of EACH

Position	Meaning
16 EACH *ingot*	Separate quantity allowance per matching resource ID.
RETAIN 64 EACH *ingot*	Separate retained-stock target per matching resource ID.
FROM EACH stock / TO EACH machines	Separate limits per selected source/destination block.
IF EACH machines HAS ...	Every selected block must satisfy the condition; alias of EVERY in a condition.

`EACH SIDE` is another fixed phrase: try all exposed side choices. It does not distribute items evenly.

Compare the resulting policy

Output	Policy
OUTPUT 8 *ingot* TO machines	Up to 8 matching ingots total across the destination scope.
OUTPUT 8 EACH *ingot* TO machines	Up to 8 per matching ingot ID across that scope.
OUTPUT 8 *ingot* TO EACH machines	Up to 8 matching ingots per destination block.
OUTPUT 8 EACH *ingot* TO EACH machines	Up to 8 of each matching ingot ID per destination block.
OUTPUT RETAIN 8 EACH *ingot* TO EACH machines	Top up each destination to 8 of each matching ingot ID.

Without block-level `EACH`, limits and retention are shared across the selected group. For example, two fuel chests with 50 coal each and `INPUT RETAIN 64 coal FROM fuel` preserve a combined reserve of 64; `FROM EACH fuel` preserves up to 64 in each chest, so neither currently supplies coal.

`EACH` by resource ID does not mean each item stack, slot, NBT variant, or enchantment combination. These distinctions and the supported combinations are illustrated in the [limits example](#).

A practical machine supply rule

```
NAME "Keep coal in every generator"

EVERY 20 TICKS DO
  INPUT RETAIN 64 minecraft:coal FROM coal_stock
  OUTPUT RETAIN 16 minecraft:coal TO EACH generators
END
```

This preserves a shared 64-coal source reserve and tries to keep 16 coal in each generator. Add the actual generator input side if necessary. Retention is a stock target, not a fairness guarantee when supplies are scarce.

8. FORGET and independent jobs

Use `FORGET` to prevent one job's sources from being used by another job.

```
NAME "Two independent routes"

EVERY 20 TICKS DO
  INPUT FROM mined_items
  OUTPUT TO ore_storage

  FORGET

  INPUT FROM harvested_crops
  OUTPUT TO food_storage
END
```

Without `FORGET`, the second output could also draw eligible leftovers from `mined_items`. That matters even when the first output inserted some items successfully.

Form	Effect
<code>FORGET</code>	Clear all active input labels for the current trigger.
<code>FORGET mined_items</code>	Remove that label from active inputs.
<code>FORGET mined_items, harvested_crops</code>	Remove both named labels from active inputs.

Forgetting does not erase the labels on the disk, delete blocks, or discard items. It changes the current execution's eligible input sources. An `IF` block does not provide a private input scope; input statements reached inside it remain available afterward until forgotten or the trigger ends. See [the built-in FORGET example](#) and [ForgetStatement](#).

9. Filtering, wildcards, and tags

Literal IDs and wildcard patterns

Use a full ID when you mean one resource: `minecraft:copper_ingot`. Use `*` in unquoted patterns for zero or more characters: `*_ingot`, `mekanism:*`, `*ore*`.

Quoted **resource filters** support Java-style regular expressions, for example `"minecraft:(iron|gold)_ingot"`. Quoted **labels** are still exact names, not regex patterns. Quote resource paths containing punctuation that the plain identifier grammar cannot accept, such as a slash. Use a literal filter when you do not need a pattern.

EXCEPT applies to the whole transfer statement

```
NAME "Keep valuables out of bulk storage"

EVERY 20 TICKS DO
    INPUT * EXCEPT minecraft:diamond, minecraft:emerald FROM incoming
    OUTPUT TO bulk_storage
END
```

Every resource-limit entry in that `INPUT` is subject to the exclusion list. Exclusions are IDs/patterns separated by commas. `EXCEPT` does not accept a `WITH` expression. The [filtering example](#) documents this statement-wide behavior.

Tags group related resources

Tags are Minecraft's named sets of resources. Exact tag availability depends on installed mods and data packs; check the tag with inspection, a recipe viewer, or the relevant data files before relying on it. For Minecraft 1.21.1, common interoperable item tags often use `c:`; older examples may use `forge:`.

```
NAME "Collect tagged ingots"

EVERY 20 TICKS DO
    INPUT WITH #c:ingots FROM incoming
    OUTPUT TO ingot_storage
END
```

These tag selectors have equivalent spellings:

- `WITH #c:ingots`
- `WITH TAG c:ingots`
- `WITH TAG #c:ingots`

You can omit the tag namespace to match any namespace: `WITH #ingots` . A tag's path can have segments: `WITH #c:ingots/iron` . A pattern such as `#c:ingots/*` selects child tag paths; `**` can match multiple remaining path segments. Block tags can also match the items representing those blocks. This behavior is specified by the [tag-matching example](#).

Combine tag requirements

```
NAME "Non-iron ingots"

EVERY 20 TICKS DO
    INPUT WITH #c:ingots AND NOT #c:ingots/iron FROM incoming
    OUTPUT TO other_ingots
END
```

`AND` requires both conditions; `OR` requires either; `NOT` negates one. Parentheses group expressions. `WITHOUT` negates the **whole** tag expression following it:

```
WITHOUT #tag_a AND #tag_b
means: NOT (has tag_a AND has tag_b)

WITH #tag_a AND NOT #tag_b
means: has tag_a AND does not have tag_b
```

`WITH` and `WITHOUT` in this release match tags. They are not a general NBT/data-component query language: there is no supported `WITH enchantment = ...` or `WHERE damage > ...` syntax.

10. IF, ELSE, and inventory conditions

```
NAME "Conditional restocking"

EVERY 20 TICKS DO
    IF supplies HAS LT 32 minecraft:iron_ingot THEN
        INPUT minecraft:iron_ingot FROM warehouse
        OUTPUT RETAIN 64 minecraft:iron_ingot TO supplies
    ELSE IF supplies HAS GT 128 minecraft:iron_ingot THEN
        INPUT RETAIN 128 minecraft:iron_ingot FROM supplies
        OUTPUT TO warehouse
    ELSE
        -- Stock is within the chosen range; no transfer is needed.
    END
END
```

Branches are checked in order; the first true branch runs. An optional `ELSE` handles the remaining case. `ELSE IF` is two keywords. Nested `IF` statements are supported.

Comparison operators

Word	Symbol	Meaning
GT	>	Greater than
LT	<	Less than
EQ	=	Equal to
GE	>=	Greater than or equal to
LE	<=	Less than or equal to

Use `=`, not `==`. There is no `!=` operator; write `NOT (supplies HAS EQ 0 minecraft:coal)` or simply use `GT 0` for a positive count.

The condition format is:

```
[OVERALL | SOME | EVERY | EACH | ONE | LONE]
labels [sides] [slots] HAS comparison number [resource alternatives] [tag filter]
```

A missing resource filter counts items by default. State the resource type explicitly when counting fluids, energy, or chemicals. A count is taken when the condition is evaluated, so do not treat a condition and a later output as an atomic crafting reservation.

Conditions on groups

Suppose three chests labelled `supplies` contain 5, 5, and 10 iron ingots.

Condition	Result	Why
<code>supplies HAS EQ 20 iron_ingot</code>	True	Default is an overall sum.
<code>OVERALL supplies HAS EQ 5 iron_ingot</code>	False	Total is 20.
<code>SOME supplies HAS EQ 5 iron_ingot</code>	True	At least one chest has exactly 5.
<code>EVERY supplies HAS GE 5 iron_ingot</code>	True	All three pass.
<code>EACH supplies HAS GE 5 iron_ingot</code>	True	Same as <code>EVERY</code> .
<code>ONE supplies HAS EQ 5 iron_ingot</code>	False	Two chests pass, rather than exactly one.
<code>LONE supplies HAS EQ 5 iron_ingot</code>	False	<code>LONE</code> allows zero or one passing chest; two pass.

With an empty selected group, `EVERY` and `LONE` can evaluate true, `SOME` and `ONE` are false, and `OVERALL` compares against zero. Do not use `EVERY ...` alone as proof that a label was assigned correctly. Group behavior is implemented in `SetOperator` and `BoolHas`.

Combine boolean conditions

```
EVERY 20 TICKS DO
  IF (supplies HAS LT 16 minecraft:coal)
    AND (warehouse HAS GT 64 minecraft:coal)
    AND REDSTONE GT 0 THEN
    INPUT RETAIN 64 minecraft:coal FROM warehouse
    OUTPUT RETAIN 32 minecraft:coal TO supplies
  END
END
```

Precedence is NOT , then AND , then OR . Parentheses make mixed expressions easier to read. TRUE and FALSE are boolean constants.

4.34.0 caveat: the grammar accepts EXCEPT in a HAS condition, but the inspected BoolHas counting implementation does not apply that exclusion field. Treat this as an implementation issue and avoid relying on it. Use explicit resource alternatives or a suitable positive/negative tag filter instead. This caveat is based on [source inspection](#), not a live-world reproduction.

11. Machine sides and slots

Sides choose a block's exposed resource interface. They do not tell SFM where to lay a cable.

```
NAME "Feed and collect a furnace"

EVERY 20 TICKS DO
  INPUT minecraft:raw_iron FROM ores
  OUTPUT RETAIN 8 minecraft:raw_iron TO furnace TOP SIDE
  FORGET

  INPUT minecraft:coal FROM fuel
  OUTPUT RETAIN 8 minecraft:coal TO furnace BOTTOM SIDE
  FORGET

  INPUT minecraft:iron_ingot FROM furnace BOTTOM SIDE
  OUTPUT TO finished
END
```

For a normal furnace, the top is the smelting-item input and the bottom exposes results and fuel handling. Other machines may expose different sides and may require their own side configuration. The [official furnace example](#) uses top and bottom in this way.

Side syntax	Meaning
TOP SIDE , BOTTOM SIDE	World up or down.
NORTH SIDE , EAST SIDE , SOUTH SIDE , WEST SIDE	Absolute world directions.

Side syntax	Meaning
FRONT SIDE , BACK SIDE	Based on the block's facing property.
LEFT SIDE , RIGHT SIDE	Relative horizontal faces. For a north-facing block, LEFT resolves east and RIGHT west in this implementation.
TOP, WEST SIDE	Consider the listed sides. The word SIDE comes once at the end.
NULL SIDE	Request the unsided interface.
No side written	Defaults to NULL SIDE , not all six faces.
EACH SIDE	Try all six directions plus the unsided interface.

Relative faces depend on the block having a supported facing property; if they cannot be resolved, the implementation falls back to an unsided query. Start with explicit world directions when diagnosing a machine. EACH SIDE is not a bypass of a machine's I/O rules, and broad side access can complicate counting. See [Side](#) and [SideQualifier](#).

Slot selection

Slot indices are **zero-based indices in the exposed handler**. A machine's displayed GUI positions may not use the same numbering.

```
EVERY 20 TICKS DO
  INPUT FROM cabinet SLOTS 0-8
  OUTPUT TO sorting_chest SLOTS 9-17
END
```

SLOT and SLOTS are aliases. Ranges include both endpoints. SLOTS 0, 2, 5-8 selects 0, 2, 5, 6, 7, and 8. With both sides and slots, write sides first: furnace BOTTOM SIDE SLOTS 2 .

For fluids/chemicals, the handler's slots correspond to exposed tanks; for other resource types they follow that type's handler. Use SFM's Container Inspector and machine settings to discover the actual indices. Retention applies to this selected view, so filtering to a single slot also limits what stock is counted.

12. Round robin and distribution

Round robin chooses a different target each time a statement executes. It does not split a batch evenly by itself.

```
NAME "Take turns supplying stations"

EVERY 20 TICKS DO
  INPUT minecraft:stone FROM warehouse
  OUTPUT 16 minecraft:stone TO stations ROUND ROBIN BY BLOCK
END
```

With three eligible station blocks, successive executions select one block, then another, then the third, cycling thereafter. The selected block can receive up to 16 stones; a full or incompatible target can prevent useful work for that turn. Do not depend on a particular first block or ordering.

Form	Selection
ROUND ROBIN BY BLOCK	Select one block across the labelled candidates.
ROUND ROBIN BY LABEL	Select one label from the label list; the blocks assigned to it are candidates.

Round robin works on inputs too:

```
EVERY 20 TICKS DO
  INPUT FROM quarry_a, quarry_b ROUND ROBIN BY LABEL
  OUTPUT TO processing
END
```

For keeping several machines stocked, `OUTPUT RETAIN 16 ... TO EACH machines` is usually a clearer policy: it visits the destination group and tops up each matching stock level. Use round robin when taking turns is the actual requirement. Selection behavior is explained in the [round-robin example](#) and [implementation](#).

13. Redstone triggers and remote signals

A pulse moves one batch

```
NAME "Button-operated batch"

EVERY REDSTONE PULSE DO
  INPUT 16 minecraft:cobblestone FROM source
  OUTPUT TO destination
END
```

Attach the button or pulse source so the **manager** receives the signal. A pulse trigger is an event; holding a lever on is not an instruction to repeat this block every tick.

A lever enables scheduled work

```
NAME "Lever-enabled transfer"
```

```
EVERY 20 TICKS DO
  IF REDSTONE GT 0 THEN
    INPUT FROM source
    OUTPUT TO destination
  END
END
```

`IF REDSTONE` checks the manager's strongest neighboring redstone signal, not an arbitrary label. Use an explicit comparison such as `GT 0` for powered, `EQ 0` for unpowered, or `GE 8` for a threshold.

4.34.0 caveat: bare `IF REDSTONE THEN` is parsed as `REDSTONE GE 0` by the inspected builder, making it true even at signal zero. Always write the intended comparison. The [redstone example](#), [builder](#), and [BoolRedstone implementation](#) establish the distinction.

Inspect a remote block's signal

```
EVERY 20 TICKS DO
  IF control HAS GT 0 redstone:: THEN
    INPUT FROM source
    OUTPUT TO destination
  END
END
```

Here `control` is a labelled block whose exposed redstone handler supplies the reading. Inspect it and choose a side if needed. `redstone::` is a resource-type selector; ordinary `minecraft:redstone` is the redstone-dust **item**. The release's redstone resource implementation reports signal values but has no-op extraction/insertion, so do not promise that `OUTPUT 15 redstone::` will act as a general redstone emitter. See [RedstoneResourceType](#).

14. Empty slots

```
NAME "Insert only into empty slots"
```

```
EVERY 20 TICKS DO
  INPUT FROM drops
  OUTPUT TO EMPTY SLOTS IN storage
END
```

`EMPTY SLOTS IN` changes destination selection: occupied slots are skipped, even if an existing stack could accept more items. This can reduce work scanning some large nearly full inventories, but can leave partly filled stacks unmerged. Compare behavior before applying it broadly.

With multiple destination blocks, the modifier order is `TO EMPTY SLOTS IN EACH storage` . Slot/side filters still come after the labels. Use ordinary output for normal stacking. The official [empty-slots example](#) also shows a separate infrequent compaction pass.

15. Keyword reference

This table covers every word token in the shipped SFML lexer, including aliases and a reserved token that has no usable statement syntax. Examples in the table are fragments unless they show a complete trigger.

Keyword or phrase	What it does	Example / note
NAME	Gives the program a title.	NAME "Ore line" ; before triggers.
EVERY	Starts a scheduled/pulse trigger; also means all blocks in a HAS condition.	EVERY 20 TICKS DO ; IF EVERY furnaces HAS ...
DO	Starts a trigger body.	Paired with END .
END	Closes a trigger or IF block.	Nested blocks need separate endings.
TICK , TICKS	Interval unit in game ticks.	EVERY 20 TICKS .
SECOND , SECONDS	Interval unit of 20 game ticks.	EVERY 2 SECONDS .
GLOBAL , G	Uses the world game-time clock.	EVERY 20G TICKS .
PLUS , +	Adds a timer phase offset.	EVERY 20 PLUS 5 TICKS ; not general math.
REDSTONE	Pulse source keyword, manager signal condition, or usable identifier in other contexts.	IF REDSTONE GT 0 ; distinguish dust from redstone::
PULSE	Selects redstone event triggering.	EVERY REDSTONE PULSE DO .
INPUT	Registers eligible sources and extraction limits.	INPUT 8 coal FROM fuel .
FROM	Introduces source labels.	FROM ore_input .
OUTPUT	Transfers eligible inputs to destinations.	OUTPUT TO finished .
TO	Introduces destination labels.	TO EACH machines .
RETAIN	Source reserve or destination target stock.	INPUT RETAIN 64 ... ; OUTPUT RETAIN 16 ...
EACH	Separates limits by resource ID or block; all-block condition alias; part of EACH SIDE .	Meaning depends on its position.
EXCEPT	Excludes resource IDs/patterns from the entire transfer statement.	EXCEPT dirt, gravel ; condition caveat in section 10.
FORGET	Clears active input references.	FORGET or FORGET source .
EMPTY	Selects initially empty output slots.	Used in TO EMPTY SLOTS IN storage .
SLOT , SLOTS	Selects handler indices; also used in the empty-slot phrase.	SLOTS 0, 2-5 .
IN	Connects empty-slot selection to destination labels.	EMPTY SLOTS IN storage .

Keyword or phrase	What it does	Example / note
WITH	Requires a tag expression.	WITH #c:ingots .
WITHOUT	Negates the following whole tag expression.	WITHOUT #c:ores .
TAG , #	Introduces a tag matcher.	TAG c:ingots , #c:ingots , or TAG #c:ingots .
IF	Starts conditional execution.	IF stock HAS LT 64 coal THEN .
THEN	Starts the true branch.	Required after the condition.
ELSE	Fallback branch or part of ELSE IF .	No THEN directly after plain ELSE .
HAS	Counts matching resources in selected blocks.	stock HAS GE 64 coal .
OVERALL	Compares the aggregate count; default HAS mode.	OVERALL tanks HAS GE 1000 fluid::water .
SOME	At least one selected block must pass.	SOME machines HAS EQ 0 coal .
ONE	Exactly one selected block must pass.	Different from SOME .
LONE	Zero or one selected block may pass.	Not the same as ONE .
TRUE	Always-true boolean value.	Useful while isolating a branch.
FALSE	Always-false boolean value.	Can temporarily prevent a branch from executing.
NOT	Negates a boolean or tag expression.	NOT (stock HAS EQ 0 coal) .
AND	Requires both boolean/tag expressions.	condition_a AND condition_b .
OR	Either boolean/tag expression; also resource alternatives sharing a limit.	8 coal OR charcoal .
GT , >	Greater than.	HAS GT 16 coal .
LT , <	Less than.	HAS LT 16 coal .
EQ , =	Equal to.	A single equals sign.
GE , >=	Greater than or equal to.	HAS GE 16 coal .
LE , <=	Less than or equal to.	HAS LE 16 coal .
ROUND	Begins the round-robin selection phrase.	ROUND ROBIN BY BLOCK .
ROBIN	Second word of that phrase.	Not a standalone statement.
BY	Selects the rotation grouping.	Followed by LABEL or BLOCK .
LABEL	Rotate between label groups.	ROUND ROBIN BY LABEL .
BLOCK	Rotate between individual blocks.	ROUND ROBIN BY BLOCK .
SIDE	Ends a side selector.	TOP SIDE , TOP , WEST SIDE , EACH SIDE .

Keyword or phrase	What it does	Example / note
TOP , BOTTOM	Absolute up/down faces.	TOP SIDE .
NORTH , EAST , SOUTH , WEST	Absolute horizontal faces.	NORTH SIDE .
FRONT , BACK , LEFT , RIGHT	Faces relative to the block's facing.	See the exact mapping in section 11.
NULL	Unsided access.	NULL SIDE ; also the default.
WHERE	Reserved lexer token, with no supported statement production in 4.34.0.	Do not use as a filter keyword.

Punctuation reference

Symbol	Purpose
--	End-of-line comment.
"..."	Quoted title, label, or resource filter.
,	List separator.
:	Separates resource ID/type or tag namespace components.
*	Wildcard in unquoted resource/tag patterns.
#	Tag introducer.
/	Tag path separator; quote ordinary resource paths that require it.
-	Inclusive slot range, as in 0-8 . Not subtraction.
(,)	Groups boolean/tag expressions; also regex syntax inside a quoted resource filter.
+	Timer phase offset.

There are no SET , VAR , LET , FUNCTION , RETURN , IMPORT , WAIT , CRAFT , FOR , FOREACH , or WHILE statements in this release. EACH is a modifier, not a loop body. The keyword inventory is checked against the [exact lexer grammar](#).

16. Ready-to-edit examples

Each file is a complete separate program. Match its label names to your network before use. Avoid running two scripts against the same inventories until you understand their combined behavior.

File	Purpose	Labels to assign
01-first-transfer.sfml	Move items between two chests.	source , destination
02-priority-and-overflow.sfml	Keep workshop iron stocked, then warehouse overflow.	incoming , workshop , warehouse

File	Purpose	Labels to assign
03-reserve-and-restock.sfml	Preserve source coal and refill each generator.	coal_stock , generators
04-furnace-line.sfml	Feed raw iron/fuel and collect ingots from furnaces.	ores , fuel , furnaces , finished
05-fluid-top-up.sfml	Keep up to one bucket of water in each destination tank.	water_source , water_tanks
06-fe-distribution.sfml	Send capped FE to each machine every tick.	battery , machines
07-chemical-transfer.sfml	Move Mekanism oxygen through configured machine faces.	oxygen_source , oxygen_users
08-tag-sorter.sfml	Route tagged ingots first, then remaining items.	incoming , ingot_storage , other_storage
09-lever-enabled.sfml	Transfer only while the manager receives power.	source , destination
10-pulse-batch.sfml	Move up to 16 cobblestone per redstone pulse.	source , destination
11-round-robin.sfml	Give individual stations alternating deliveries.	warehouse , stations
12-slot-routing.sfml	Move between selected rows of ordinary chests.	cabinet , sorting_chest
13-conditional-restocking.sfml	Refill low stock and return excess stock.	supplies , warehouse
14-independent-jobs.sfml	Keep two routes separate with FORGET.	mined_items , ore_storage , harvested_crops , food_storage
15-resource-and-block-each.sfml	Maintain each ingot type separately in each supply chest.	warehouse , workbenches
16-empty-slot-insertion.sfml	Insert only into empty destination slots.	drops , storage

The exact source/destination machine interfaces remain your responsibility: a syntactically valid script cannot turn an output-only port into an input port. These examples intentionally do not assume an entire production recipe, universal slot numbering, or unlimited throughput.

17. Working with other mods

These integrations are optional. The corresponding mod must be installed, and its blocks must expose a compatible resource handler. Core SFML examples using ordinary chests and furnaces require none of these integrations.

Mekanism factories and chemical lines

Select the correct resource type independently for items, FE, fluids, and chemicals. Configure the relevant **resource category** on each machine side. SFM's **INPUT** extracts from the machine's **output** face; SFM's **OUTPUT** inserts into the machine's **input** face. If possible, inspect the machine and use the suggested sides. Mekanism's item auto-sort feature is a separate machine setting; **TO EACH factories** distributes by block, not by processing lane.

An oxygen routing example is included, with **EAST SIDE** and **WEST SIDE** as explicit example faces. Change the machine settings or those directions to match your build. The [Mekanism compatibility code](#) includes side inspection and chemical registration.

Create processing

SFM routes resources to compatible inventories/tanks. It does not execute a `CRAFT` instruction, supply rotational power, or simulate the machine's recipe. The actual machine must perform its processing. Slots and sides can differ between a depot, basin, chest, and another mod's processing block.

Storage drawers, AE2, and Refined Storage

Target an appropriate exposed inventory/interface block and confirm its behavior with a small item sample. A visible storage GUI is not proof that every network item is exposed through that block's external item handler. Filtering and retention count what the selected handler exposes. Pattern-based autocrafting and SFM routing are separate mechanisms.

Visual editors and generated scripts

If a visual editor generates SFML, inspect the resulting program when diagnosing a problem. An editor may expose only part of the language or target a different release. SFM's own parser and problems view are the version-specific authority for whether that program is accepted.

Moving or rebuilding machines

When blocks are moved, replaced, or assembled by another mod, recheck the manager's label assignments, cable connectivity, and the machines' exposed resource handlers. Test a small transfer after the change before relying on a complete factory. Moving-structure support depends on the participating mods and is not established by SFML syntax alone.

18. Troubleshooting

Symptom	What to check
Program will not compile	Missing <code>END</code> , missing <code>DO</code> / <code>THEN</code> , <code>==</code> instead of <code>=</code> , unquoted reserved labels, wrong modifier order, or unsupported keywords.
Minimum trigger interval error	Use at least the configured interval; separate FE-only I/O into its own trigger.
Program runs but moves nothing	Labels pushed to the correct manager, cable network, loaded chunks, compatible handlers, source contents, and destination capacity.
Items work but water/energy does not	Explicit resource type on both INPUT and OUTPUT: <code>fluid::</code> , <code>fe::</code> , or <code>chemical::</code> .
Wrong items end up in a machine	A broad input/output filter, source inputs left active, overlapping labels, or missing <code>FORGET</code> .
Only one machine gets resources	A shared quantity cap, full destinations, source shortages, or round robin. Use block-level <code>EACH</code> where appropriate.
Machine keeps accumulating more fuel	<code>OUTPUT 16</code> adds another batch every run. <code>OUTPUT RETAIN 16</code> maintains a target.
Matching stock is not being counted as expected	Resource namespace, tags, selected sides/slots, hidden machine compartments, and overlapping label groups.
Filter matches no ore/ingot	Registry ID differs from display name, tag not present, or a tag namespace from a different Minecraft version was copied into the program.
Chemical transfer fails	Fluid versus chemical type, exact chemical ID, appropriate resource-category side settings, and actual external tank access.
A lever-off program still runs	Check for bare <code>IF REDSTONE</code> ; use <code>IF REDSTONE GT 0</code> .

Symptom	What to check
A timed job never runs	An offset outside the interval, unloaded manager, compilation problems, or program execution disabled by server settings.
A group test is unexpectedly true	Missing labels can leave an empty group; <code>EVERY</code> / <code>LONE</code> may still be true.
Old network stopped updating	Check SFM's "Rebuild cable network" control after confirming physical connectivity.
A composter accepts unexpected items	Specify a valid insertion side; the built-in known-issues example warns that unsided insertion can behave like a trash can in affected versions.

The cable-cache and composter notes come from the release's [known-issues example](#). Use the smallest network that reproduces a problem, then add one rule at a time. Log/inspection tools help distinguish a parser error from a label, filter, or external-machine issue.

A safe way to develop a program

1. Prove an unfiltered chest-to-chest transfer works.
2. Add one exact item filter.
3. Add the real machine with one explicit side.
4. Add stock targets and per-block `EACH` if needed.
5. Separate additional jobs with `FORGET` or independent triggers.
6. Test low source stock, a full destination, and a temporarily unavailable machine.
7. Check that routing remains sensible after the relevant chunks unload/reload.

19. Quick reference

```
PROGRAM
  NAME "Optional title"
  EVERY 20 TICKS DO
    statements
  END

TRANSFER
  INPUT [amount [EACH]] [RETAIN amount [EACH]] [resource filter]
    [WITH tag expression] [EXCEPT resources] FROM [EACH] labels
  OUTPUT [amount [EACH]] [RETAIN amount [EACH]] [resource filter]
    [WITH tag expression] [EXCEPT resources] TO [EACH] labels

AFTER LABELS, IN THIS ORDER
  [ROUND ROBIN BY BLOCK | ROUND ROBIN BY LABEL]
  [TOP SIDE | other side selector]
  [SLOTS 0, 2-5]

CONDITIONAL
  IF [OVERALL | SOME | EVERY | EACH | ONE | LONE]
    labels HAS comparison amount resource THEN
      statements
  ELSE IF another condition THEN
    statements
  ELSE
    statements
  END

REMEMBER
  INPUT = eligible sources; OUTPUT = actual transfer attempt
  INPUT RETAIN = preserve source reserve
  OUTPUT RETAIN = top up destination stock
  8 EACH resource = per resource ID
  TO EACH label = per block
  FORGET = clear active source references
  fe:: / fluid:: / chemical:: = explicit resource types
  -- = comment; # = tag
```

This is notation, not a complete pasteable program. Download a file from section 16 when you want a starting script.

20. Sources and validation

Primary references are pinned to the SFM 4.34.0 release used for validation. The manual is original explanatory material, not a replacement for the mod author's source or issue tracker.

- Release: [4.34.0-1.21.1](#).
- Parser grammar: [SFML.g4](#).
- Author's built-in examples: [template programs](#).
- Language implementation: [AST nodes](#).
- Transfer accounting: [program trackers](#).
- Mod source is licensed under [MPL-2.0](#).

The companion [VALIDATION.md](#) records how the example syntax and documentation were checked. Parser acceptance does not prove live inventory behavior, machine configuration, network reachability, or moving-vehicle integration. Implementation caveats above are identified as source findings where they were not reproduced in-game.

AI-generated independent guide · SFM 4.34. Prepared September 13, 2026. Works offline; no external scripts, fonts, or analytics. [Package information](#) · [Validation details](#)